

MCP DEV SUMMIT TORONTO · TUE 6 OCT · 11:50-12:15 · MCP DEMO V3.0.1

MCP as a Platform

What I Learned Building a Portfolio of MCP Servers

One server teaches you the protocol. A portfolio teaches you what MCP actually is — and the mistakes I only found by building the next one.

Every example here is in one small server you can clone tonight: **MCP Demo v3.0.1** — github.com/markusvankempen/mcp-ticket-demo

Markus van Kempen · Mensch, Maker and Developer
No bug too small, no syntax too weird.

mvankempen@ca.ibm.com

markus.van.kempen@gmail.com

[Slides online](#)

markusvankempen.github.io



Slides online

THE RUNNING EXAMPLE · DEMO

Meet the ticket server.

Every example today runs on it.

```
# in the IDE chat
> Show me all tickets and their status.

→ run_query schema: "tickets"
← TCK-1001 open      0 tools discovered after I registered the server
   TCK-1002 pending  Customer never got the reply – ticket looks fine in the API
   TCK-1003 solved   mcp.json worked yesterday, 404 today
```

Local: localhost:8787 · started with `MCP_MODE=http npx mcp-ticket-demo` · tickets are only the stand-in

Every seed ticket is a mistake from this talk.
The rest of the talk is how each one got fixed.

HOW THE PLATFORM GREW

Every answer raised a new question.

01

Prove it's up

02

Easy to ship and use

03

Which server answers?

04

When OK wasn't OK

05

Past 100 tools

06

A control plane

07

Remote configuration

08

The server talks back

Running example: **mcp-ticket-demo**. Tickets are only the stand-in.

WHERE IT STARTS

The first version is deceptively simple.

```
client → MCP server → your API
```

```
|  
+-- tool: request(path, method, query, body)
```

That is a working MCP server. Then the real requests arrive:

```
create one, update one, close one  
a safer default than "return everything"  
what "no results" means, so it stops retrying  
who is allowed to call the write tools  
and how anyone finds the server at all
```

SDK

MCP TypeScript SDK

`McpServer` plus the stdio,
Streamable HTTP and SSE transports.
1.30.

SCHEMAS

Zod

Input and output schemas. A bad call
is refused before your code runs.

HTTP

Express 5

`/mcp`, `/health`, `/test` and
`/admin` on one port.

SHIP

npm · registry · vsce

`npm publish`, `mcp-publisher` for
the MCP Registry, `@vscode/vsce` for
the extension.

WHERE IT STARTS · NAMES

With a few tools, names barely matter. With dozens, the name is all the model has.

Ticketing is just the stand-in. Any API works the same way — orders, assets, containers. For this talk the stand-in is a small support-ticket system, packaged as **MCP Demo**.

The early question

"Can the model call this API?"

The later question

"Can a developer understand and operate this tool safely?"

```
search_tickets(query, status, requester_email, limit)
```

is easier to use — and safer — than:

```
request(path, method, query, body)
```

The second one is less code for me
and every decision for the model.

01 · PROVE IT'S UP · DEMO

The IDE says zero tools. Ask the server.

HOST = `http://localhost:8787` · started with `MCP_MODE=http npx mcp-ticket-demo`

```
# what the IDE shows
```

```
0 tools discovered
```

```
No error. No warning.
```

```
$ curl -s $HOST/health?format=json
```

```
{ "ok": true,  
  "version": "3.0.1",  
  "tools": 19 }
```

The client can't tell you why.
The server's count can: the problem is the connection.

Try it: `curl -s localhost:8787/health?format=json`

01 · ALIVE IS NOT WORKS · DEMO

Alive is not works. Ask the tools, not the process.

```
# the MCP standard: ping
```

```
→ { "method": "ping" }
```

```
← { "result": {} }
```

The process answered.

That is all it says.

```
$ curl -s $HOST/test?format=json
```

```
{ "ok": true, "steps": [  
  { "name": "tickets (open)",  
    "ok": true },  
  { "name": "run_query tickets",  
    "ok": true } ... ] }
```

Ping proves the process is alive.
Only calling the tools proves they work.

Try it: `curl -s localhost:8787/test?format=json` · `curl -s localhost:8787/tools?format=json` · then `/help` and `describe_server`, the tool to call first.

From curl and guesswork to one install.

Editor extension

Ships the server
Writes every mcp.json
Diagnoses the chain



MCP server

Same tools
Same schemas
Same domain behaviour



Enterprise API

Tickets · Assets
Jobs · Containers
Agents

Transports

studio

Streamable HTTP

SSE

Runs on

laptop · npx

container · Podman

cloud · Code Engine

```
# before: curl and guesswork
curl $HOST/health
curl $HOST/tools?format=json
edit mcp.json, restart: 0 tools
```

MCP Platform diagnostics

http://127.0.0.1:8787

Setup

Diagnose

MCP Test

Settings

Chat

CHAT → SERVER

VS Code ·
http (shared)

LOCAL PODMAN

not used

REMOTE HTTP

not used

NPM UPDATE

current

DIAGNOSE

PASS

Open panel

/health

/test

/admin

/tools

/help

Resources

Config present, server alive and the same version as the code on disk, chat pointed at that server, tools/list, one real call.

The extension is the front door.
The MCP server is the capability.

Which server is the client actually talking to?

Two memories

Each IDE's stdio config spawns its own copy.

A stale copy

`npx ...@latest` in the IDE, a bundled copy in the extension.

A stranger on the port

Another server answered. Its tools showed up as mine.

FIX

HTTP by default

The extension and every `mcp.json` point at one server. One process, one memory.

```
# every copy says which copy it is
{ "service": "mcp-ticket-demo",
  "instance": {
    "label": "local",
    "id": "Markuss-MacBook-Pro.local",
    "url": "http://127.0.0.1:8787" } }
```

Same `instance` on `/health`, `/test`, `/tools` and `describe_server`. The extension checks it before trusting the tool list.

04 · WHEN OK WASN'T OK

The call succeeded. The outcome was wrong.

201 Created

Owner: `mcp-bot@service.local`
The customer never heard back.

403 Forbidden

The model retried. And retried.
"The system is unavailable."

A tool isn't done when the API call succeeds.
It's done when the next thing that happens is right.

04 · HOW WE STOPPED IT

Tell the model before. Tell it again after.

```
# the tool description (before the call)
```

```
create_ticket
"Open a support ticket.
ALWAYS pass requester_email -
the real customer's email.
If you leave it out, the bot
owns the ticket."
```

```
# the tool result (after the call)
```

```
{ "ok": true, "status": 201,
  "warning": "The bot owns this
             ticket. Replies go to the bot.",
  "next": "Call create_ticket again
           with requester_email set." }
```

The description stops the mistake.
The result catches the one that slips through.

04 · ERRORS THAT TEACH

A wrong call should teach the right one.

```
# 403 Forbidden
```

- the model tweaks a field
- tries again, and again
- "the system is unavailable"

```
{ "ok": false, "status": 403,  
  "error": "key has scopes [read]  
    but create_ticket requires  
    the write scope. Issue a key  
    with that scope on /admin,  
    then retry once." }
```

A bare refusal starts a loop.
A refusal that names the fix ends it.

More tools didn't mean more capability.

– BEFORE –

```
query_records  
query_with_filter  
query_with_fields  
query_object_structure  
query_work_orders  
query_assets  
query_locations ...
```

Every name correct.
The model still picked wrong.

– AFTER –

```
describe_server what am I allowed to do?  
get_schema      what can I query?  
get_schema name what shape do I need?  
run_query       ask the question
```

Three tools instead of seven.
Selection stopped being a coin flip.

Technically precise. Operationally confusing.

Same fix in Code Engine MCP: about 112 tools to 28, one per noun with an `action` field. In this demo: 31 tools to 19. Backup slide after the close.

Think of it like a database.

Reads are queries. Writes stay verbs.

≈ CATALOG

What exists?`describe_server``get_schema`

≈ SELECT

Read it`run_query``tickets · customers · trace`

≈ INSERT · UPDATE

Change it`create_ticket``add_comment``close_ticket`

≈ ALTER · GRANT

Operate it`update_settings``issue_api_key`

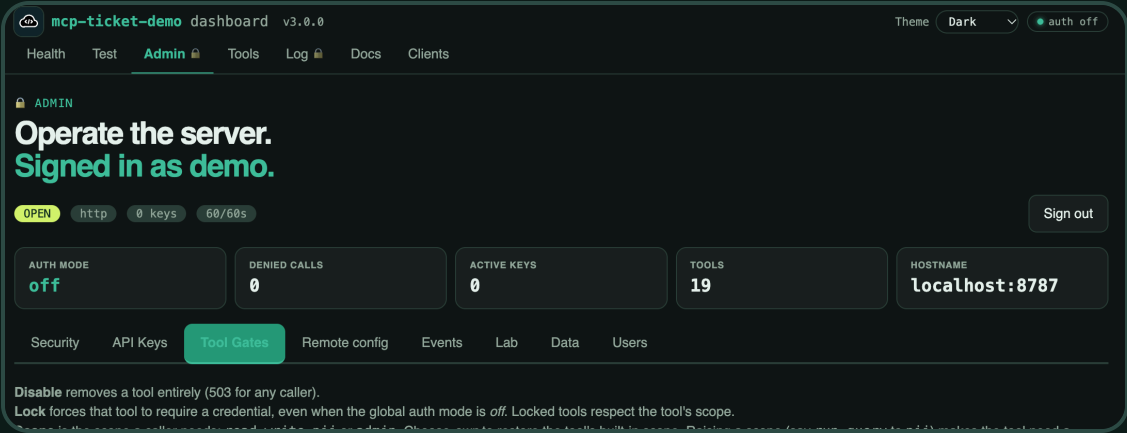
One query tool grows with schemas.
Each write keeps its own name, scope and warning.

The server got its own UI. Then it became the control plane.

/health /test /tools
see it, try every tool

/admin
keys · scopes · gates

/log
audit trail · call trace



WHY

Not everyone has the IDE

Ops, security and support open a URL.
Nothing to install.

WHY

It lives with the server

A hosted copy has no editor next to it. Its
UI runs where it runs.

WHY

One source of truth

Keys, gates and the trace live on the
server, for every client.

Configure the server remotely.

Every button is also a tool.

on /admin

Change the auth mode
Issue an API key
Add a user
Read the call trace

the same, as tools

server_access
server_api_keys
server_users
server_log

WHY

Fleet management

One script makes the same change on every copy: local, staging, production.

WHY

Incident response

Revoke a key or lock a tool everywhere, in seconds.

WHY

Agents run ops

An ops agent rotates keys and reads the trace. No browser.

Same scopes. One audit trace. No browser, no redeploy.

From a server you call to a server that tells you.

POLL

The agent keeps asking

"Anything new?" on a loop. Most answers are no, and every check costs tokens.

PUSH

The server tells you

One open stream. `ticket.*` events arrive when they happen.

```
# the server says  
ticket.created  
"Checkout is down"  
Third in ten minutes.
```

```
# an agent reacts  
run_query tickets "down"  
add_comment on all three:  
"Known incident, on it."
```

```
# a human is paged  
"Three outage tickets."  
One incident?"  
→ confirms, takes over.
```

Agents that act when something happens,
not only when someone asks.

WHAT CARRIES TO THE NEXT SERVER

MCP servers can become more than integrations.

On your next server:

01

Prove it. `/health` with a tool count, and a `/test` that calls the tools.

02 · 06

Give it front doors. Bundle the server in an IDE extension (one install, one-click setup, Diagnose), and a UI for the people who operate it.

05

Make it discoverable. `describe_server` first, `get_schema` for the shape, tool names that say the intent.

All three are in MCP Demo v3.0.1. Clone it and copy what you need.

The extension and the MCP UI are the control plane.
The MCP server is the capability.
The agent is the operator.

THANK YOU



Build one. Publish it. Make the next one easier to use.

Start with **MCP Demo**. Clone it, break it, read the refusals — every example in this talk is in there.

[MCP Demo](#) — every example from this talk · [wxo-labs](#) — 14 hands-on labs, two on MCP
No bug too small, no syntax too weird.

QUESTIONS?

mvankempen@ca.ibm.com

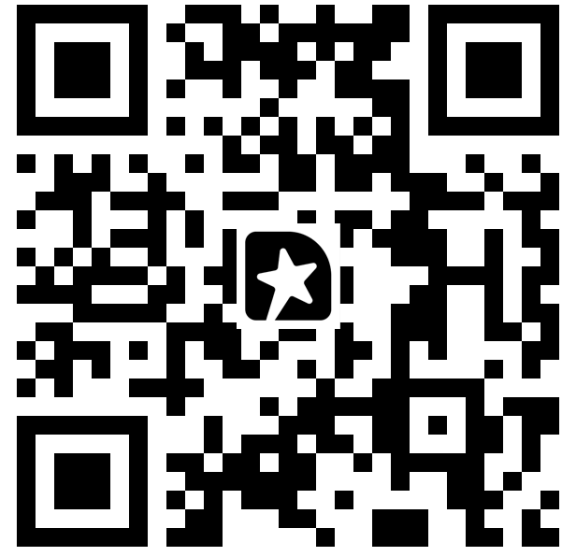
markus.van.kempen@gmail.com

[Slides online](#)

markusvankempen.github.io

Markus · Research · Floor 7½ 🏢👉

Backup slides follow — press → for acronyms, publish, deploy, auth, the curl demo, testing, and how to run the demo tonight.



MCP as a Platform: What I Learned Building a Portfolio of MCP Servers

[Scan for session feedback](#)

The acronyms.

What they actually mean.

Term	Stands for	Why it shows up in this talk
MCP	Model Context Protocol	The open standard. Clients discover and call tools; servers wrap APIs.
WXO	watsonx Orchestrate	IBM's agent orchestration platform. Labs 7–8; two servers in the portfolio.
CRN	Cloud Resource Name	IBM Cloud's globally unique ID (<code>crn:v1:...</code>). New project → new CRN → often a new hostname.
API	Application Programming Interface	The enterprise system behind the MCP server. A <code>200</code> is not a finished tool.
ADK	Agent Developer Kit	How WXO imports an MCP server as a toolkit.
CE	Code Engine	IBM Cloud serverless containers. The "cloud containers" domain.
IAM	Identity and Access Management	Why a missing tenant header fails every job while auth still looks "fine."

How MCP actually talks.

Term	What it is	Where it runs
JSON-RPC 2.0	The message format MCP speaks. Identical on every transport.	Both
stdio	The client launches your server as a child process and talks over stdin / stdout.	Local only — no port, no URL . Secrets come from env vars.
SSE Server-Sent Events	<code>text/event-stream</code> over plain HTTP. Long-lived <code>GET</code> carries server→client; a separate <code>POST</code> carries client→server.	Remote — whatever port your HTTP server binds; 443 over HTTPS in production.
Streamable HTTP	The current remote transport. One endpoint, and a response can upgrade into an SSE stream. Supersedes the older HTTP+SSE pair.	Remote — same HTTPS port.
<code>tools/list</code>	The call a client makes to enumerate what a server offers.	Both — "0 tools discovered" means this came back empty.
stderr	Where diagnostics go on stdio. <code>stdout</code> is the protocol.	Local — <code>console.log</code> corrupts JSON-RPC. Use <code>console.error</code> .

Packaging, config and distribution.

Term	What it is	Why it shows up in this talk
<code>mcp.json</code>	Client config naming a server, its command or URL, and its env vars.	Lives in <code>.vscode/</code> or <code>.cursor/</code> . The extension writes it for you.
<code>npx</code>	Runs an npm package without installing it first.	The shortest path from "found it" to "called it".
package root	Zips your server directory and uploads it so a cloud runner executes from a relative path.	The fix for 0 tools discovered (Appendix · The messy parts).
<code>/health</code> <code>/test</code> <code>/admin</code>	Is it alive, does it really work, can it be operated — <i>my convention, not the spec.</i>	What a deployed server needs to explain itself.
scope	A label on a credential — <code>read</code> , <code>write</code> , <code>pii</code> — naming what that credential may do.	How a server refuses with a reason instead of a bare <code>403</code> (Appendix · 403).
MCP Registry	The community index of published servers.	One of the places to be findable (Appendix · Publish).
Open VSX	Vendor-neutral extension marketplace for VS Code-compatible editors.	Where extensions land outside the Microsoft marketplace.

The short words.

cwd, env, PII, and the rest.

Term	Stands for	Why it shows up in this talk
<code>cwd</code>	current working directory	The folder the process starts in. Appendix · The messy parts: set it to the package you shipped, not a path from your laptop.
<code>env</code>	environment variables	How a local stdio server gets secrets. Same checks belong on every server — slide 18.
<code>npm</code>	Node Package Manager	Where the code is hosted. Publish here first, then the registry name.
<code>.vsix</code>	Visual Studio extension package	The file you upload to Open VSX. Not the registry, not npm.
<code>PII</code>	Personally Identifiable Information	A scope in the demo. Without it, a customer’s phone comes back as <code>REDACTED</code> : same call, fewer fields.
<code>CI</code>	Continuous Integration	Why a remote URL matters: a pipeline cannot use your laptop’s stdio path.
<code>IDE</code>	Integrated Development Environment	VS Code, Cursor, Windsurf, Bob — four copies of <code>mcp.json</code> (Appendix · The messy parts).
<code>HTTPS</code>	HTTP over TLS (port 443)	What a shared remote server should speak. The hostname is not something you paste by hand.

Clone the repo. Run it in about ten minutes.

Every example in this talk is in **MCP Demo**. Personal open-source project. Slides: [linuxfoundation-mcp-dev-summit](#) · You need Node 18+.

```
git clone https://github.com/markusvankempen/mcp-ticket-demo
cd mcp-ticket-demo/server
npm install
npm run http
```



Scan for the repo

Is it alive?

Open `http://127.0.0.1:8787/health`

Does it work?

Open `http://127.0.0.1:8787/test` —
it creates and finds a ticket.

Can you operate it?

Open `http://127.0.0.1:8787/admin` —
login `demo` / `demo`.

Run the tests: `npm run test:all` from `server/` (or `mise run test-all`; mise pins Node 22). Editor path: in another terminal, `npm run studio` from `server/`. Load the extension from `extension/`. Next slide is four things to try.